

File Storage, Sync, and Snapshot Design

Separate durable data bytes from metadata that names and authorizes them. Large-file systems need partial-upload handling, immutable versions, conflict rules, and garbage collection that never removes live data. A fast happy-path upload is not enough to establish durability or safe multi-device behavior.

For: Engineers interested in storage and cloud infrastructure who want to explain large-object upload, versioning, synchronization, snapshots, and safe reclamation using public generic concepts.

Suggested pace: 2-3 weeks of metadata and recovery exercises | Database and distributed-system basics

What you will be able to do

- Separate object data, metadata, versions, and access policy.
- Design resumable uploads and atomic publication of complete content.
- Handle concurrent edits, deletion tombstones, and offline synchronization.
- Explain snapshot references and conservative garbage collection.

Build the right mental model

Metadata commits make data visible

Upload chunks or parts first, validate them, and publish a complete manifest through an authoritative metadata operation. Partially uploaded bytes should not appear as a finished file. Stable operation IDs make uncertain completion requests reconcilable.

Identity is not a path

A stable file ID survives rename and move operations, while a path is mutable metadata. Version numbers or conditional writes can detect concurrent edits. Arbitrary binary conflicts should preserve competing versions rather than pretend a universal merge is possible.

Reclamation needs all references

A chunk can be referenced by the current file, history, snapshots, or restored copies. Deletion is safe only when no supported live reference remains. Grace periods and reconciliation can prefer temporary space leaks over irreversible data loss.

Study with evidence.

Use small synthetic files and a local object directory plus a metadata database. Simulate interrupted uploads and conflicting versions. State crash-consistent versus application-consistent snapshot expectations rather than assuming they are identical.

Week 1: Upload and metadata contract

Define upload IDs, part checksums, ownership, completion, expiry, and file manifests. Add an expected-base-version condition to metadata changes. Keep authorization at the authoritative read and write boundaries.

Deliverable and check: A protocol that never exposes an incomplete upload as a committed file.

Week 2, first half: Sync and conflict handling

Model per-device cursors and an ordered change log. Test rename, offline edit, duplicate completion, and deletion. Retain tombstones for a documented offline window and require re-bootstrap after it expires.

Deliverable and check: A multi-device timeline showing how conflicts are preserved and deleted files are not accidentally resurrected.

Week 2, second half: Snapshots and restore

Represent a snapshot as an immutable reference to a versioned manifest or mapping root. Use copy-on-write for later changes. Explain what is included in the consistency boundary and how restore references existing data.

Deliverable and check: A reference graph that makes shared data and ownership visible.

Week 3: Repair and reclamation

Add orphan-upload cleanup, checksum validation, corruption reporting, and safe reference-based reclamation. Estimate metadata and changed-data growth separately. Test a crash during cleanup and a restore racing with deletion.

Deliverable and check: A conservative cleanup policy and a recovery test that never deletes referenced bytes.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

Commit a file without exposing partial data

A client uploads a file as three chunks and then sends a completion request. The last chunk upload succeeds, but the completion response is lost. Another device is waiting to see the new version.

A worked approach

1. Assign an upload ID and expected file base version. Store verified chunk identities and checksums without marking the file version current. Reject completion if any required chunk is missing or the caller lacks ownership.
2. Atomically commit a manifest referencing all three chunks and record the completion operation result. The retry uses the same operation ID and receives the already committed version instead of creating a second logical update.
3. Publish the change-log entry with the metadata transaction or through a reliable outbox. The second device reads a complete manifest and can fetch chunks. Orphan cleanup only considers uncommitted uploads after the retention policy permits it.

Expected result

The new version is either absent or fully committed from a reader's perspective. Data transfer and metadata visibility have distinct boundaries, and response loss does not imply the version failed to commit.

Verify the result

- Can a manifest reference a missing or unverified chunk?
- Does duplicate completion return the same result?
- Can cleanup race with a live version or snapshot reference?

Extend the exercise

Let two devices update the same base version. Use conditional metadata writes to detect the conflict and preserve both versions or an explicit conflict copy rather than silently discarding one user's work.

Make the learning stick.

Is a snapshot always application-consistent?

No. A crash-consistent snapshot captures storage state at a defined boundary, while application consistency may require quiescing or coordinating the application. State which contract is provided and what recovery the application must perform.

Does content hashing remove all storage complexity?

No. Hashes support identity and integrity checks, but authorization, reference accounting, versioning, retention, encryption, and collision handling still require design. Cross-tenant deduplication can also create privacy concerns.

Why keep deletion tombstones?

An offline device needs evidence that a file was deleted rather than merely absent from a recent listing. Tombstones prevent accidental resurrection within the supported offline window; older devices may need a full resynchronization.

Avoid these traps

- Publishing metadata before required data is durable and verified.
- Using file paths as the only stable identity.
- Collecting data still referenced by history, snapshots, or restores.

Ready for the next step?

- Upload, commit, and visibility boundaries are distinct.
- Concurrent edits and offline deletion have explicit policies.
- Snapshots state their consistency guarantees.
- Reclamation is tested against all live references.

Reliable next reads

- [Azure Blob Storage introduction](#)
- [Azure event sourcing pattern](#)
- [OSTEP persistence chapters](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.